

Standard Operating Procedure (SOP)

K3s Installation and NGINX Web Application Deployment on Rocky Linux 9

Document Control

Item	Details
Document Title	K3s Installation and NGINX Deployment SOP
Version	1.0
Prepared By	Hemanth Raju
Date	June 2026
Platform	Rocky Linux 9
Kubernetes Distribution	K3s

1. Purpose

This document provides a step-by-step procedure for installing K3s (Lightweight Kubernetes) on Rocky Linux 9, deploying an NGINX web application, exposing it through a NodePort service, and customizing the application landing page using a ConfigMap.

2. Scope

This SOP applies to system administrators, DevOps engineers, and Kubernetes administrators responsible for deploying and managing K3s clusters for development, testing, and lab environments.

3. Prerequisites

- Rocky Linux 9 server
- Internet connectivity
- User account with sudo privileges
- Minimum 1 vCPU and 1 GB RAM (2 GB recommended)

4. Verify System Requirements

Verify operating system information:

```
cat /etc/os-release
```

Verify CPU and memory resources:

```
nproc  
free -h
```

5. Install K3s

Install K3s using the official installation script:

```
curl -sfL https://get.k3s.io | sh -
```

Expected outcome:

- K3s binary installed
- Container runtime (containerd) installed
- Kubernetes cluster initialized
- K3s service enabled and started

6. Verify K3s Installation

Check K3s service status:

```
sudo systemctl status k3s
```

Expected status:

```
Active: active (running)
```

Verify node registration:

```
sudo /usr/local/bin/kubectl get nodes
```

Expected output:

NAME	STATUS	ROLES	AGE	VERSION
localhost.localdomain	Ready	control-plane	XXm	v1.xx.x+k3s1

7. Configure kubectl Access for Non-Root User

Create kubeconfig directory:

```
mkdir -p ~/.kube
```

Copy K3s kubeconfig:

```
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
```

Set ownership:

```
sudo chown $USER:$USER ~/.kube/config
```

Verify access:

```
kubectl get nodes
```

8. Deploy NGINX Application

Create deployment:

```
kubectl create deployment nginx-web --image=nginx
```

Verify pod creation:

```
kubectl get pods
```

Expected result:

```
nginx-web-xxxxxxxxxx-xxxxx 1/1 Running
```

9. Expose NGINX Service

Expose deployment using NodePort:

```
kubectl expose deployment nginx-web --type=NodePort --port=80
```

Verify service creation:

```
kubectl get svc
```

Example:

NAME	TYPE	CLUSTER-IP	PORT(S)
nginx-web	NodePort	10.43.x.x	80:31893/TCP

10. Access the Application

Retrieve node IP address:

```
hostname -I
```

Access the application from a browser:

```
http://<NODE-IP>:31893
```

Expected page:

```
Welcome to nginx!
```

11. Create Custom HTML Content

Create a working directory:

```
mkdir -p ~/k3s-nginx-app  
cd ~/k3s-nginx-app
```

Create the custom web page:

```
vi index.html
```

Sample HTML:

```
<html>  
<head>  
  <title>My K3s App</title>  
</head>  
<body>  
  <h1>Hello from K3s</h1>  
  <p>This is my custom web page deployed using Kubernetes.</p>  
</body>  
</html>
```

Save the file.

12. Create ConfigMap

Create a ConfigMap using the HTML file:

```
kubectl create configmap nginx-html --from-file=index.html
```

Verify:

```
kubectl get configmap
```

13. Modify Deployment to Use ConfigMap

Create or update deployment YAML:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-web
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx-web
  template:
    metadata:
      labels:
        app: nginx-web
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
          volumeMounts:
            - name: html
              mountPath: /usr/share/nginx/html/index.html
              subPath: index.html

      volumes:
        - name: html
          configMap:
            name: nginx-html
```

Apply the deployment:

```
kubectl apply -f deployment.yml
```

14. Restart Deployment

Restart pods to apply ConfigMap changes:

```
kubectl rollout restart deployment nginx-web
```

Verify pod status:

```
kubectl get pods
```

15. Update Web Page Content

Modify HTML file:

```
vi index.html
```

Update the ConfigMap:

```
kubectl create configmap nginx-html  
--from-file=index.html  
-o yaml  
--dry-run=client | kubectl apply -f -
```

Restart deployment:

```
kubectl rollout restart deployment nginx-web
```

Refresh the browser to verify updated content.

16. Verification Commands

Verify cluster:

```
kubectl get nodes
```

Verify deployments:

```
kubectl get deployments
```

Verify pods:

```
kubectl get pods
```

Verify services:

```
kubectl get svc
```

Verify ConfigMaps:

```
kubectl get configmaps
```

View all resources:

```
kubectl get all
```

17. Troubleshooting

Kubeconfig Permission Error

Error:

```
error loading config file "/etc/rancher/k3s/k3s.yaml": permission denied
```

Resolution:

```
mkdir -p ~/.kube  
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config  
sudo chown $USER:$USER ~/.kube/config
```

ConfigMap Already Exists

Error:

```
configmaps "nginx-html" already exists
```

Resolution:

```
kubectl create configmap nginx-html  
--from-file=index.html  
-o yaml  
--dry-run=client | kubectl apply -f -
```

NodePort Not Accessible

Verify service:

```
kubectl get svc
```

Check firewall status:

```
sudo systemctl status firewalld
```

Allow NodePort range if required:

```
sudo firewall-cmd --add-port=30000-32767/tcp --permanent  
sudo firewall-cmd --reload
```

18. Conclusion

K3s was successfully installed on Rocky Linux 9. An NGINX application was deployed, exposed through a NodePort service, and customized using a ConfigMap. The deployment was validated through browser access, confirming successful end-to-end functionality of the K3s environment.